LEMBAR KERJA PRAKTIKUM (LKP)

MODUL 8

Array Satu Dimensi dan Sequential Search

Identitas	Isian
Nama	Muhammad Faiz Qodri Muslim
NIM	2610511179
Kelas	E
Tanggal Praktikum	21 October 2026

Petunjuk Pengerjaan

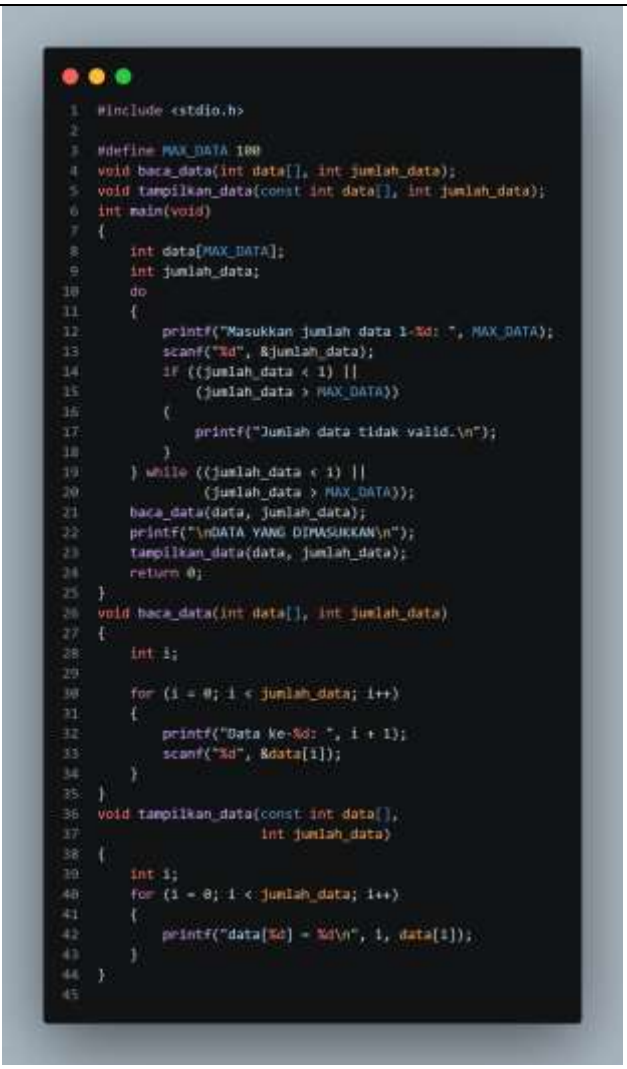
Gunakan Modul 8 sebagai panduan utama. Ketik source code sendiri pada editor/IDE. LKP digunakan untuk mendokumentasikan proses, hasil, tracing, debugging, pengujian, dan refleksi. Jangan menyalin source code lengkap ke dalam LKP; lampirkan tangkapan layar source code dan output sesuai bagian yang diminta.

IV. Praktikum Terbimbing

1. Praktikum Terbimbing 1 — Input dan Output Array

Kerjakan program latihan_01.c sesuai Modul 8.

Tangkapan layar source code latihan_01.c



```c
#include <stdio.h>

#define MAX_DATA 100
void baca_data(int data[], int jumlah_data);
void tampilkan_data(const int data[], int jumlah_data);
int main(void)
{
    int data[MAX_DATA];
    int jumlah_data;
    do
    {
        printf("Masukkan jumlah data 1-%d: ", MAX_DATA);
        scanf("%d", &jumlah_data);
        if ((jumlah_data < 1) ||
            (jumlah_data > MAX_DATA))
        {
            printf("Jumlah data tidak valid.\n");
        }
    } while ((jumlah_data < 1) ||
             (jumlah_data > MAX_DATA));
    baca_data(data, jumlah_data);
    printf("\nDATA YANG DIMASUKKAN\n");
    tampilkan_data(data, jumlah_data);
    return 0;
}
void baca_data(int data[], int jumlah_data)
{
    int i;

    for (i = 0; i < jumlah_data; i++)
    {
        printf("Data ke-%d: ", i + 1);
        scanf("%d", &data[i]);
    }
}
void tampilkan_data(const int data[],
                    int jumlah_data)
{
    int i;
    for (i = 0; i < jumlah_data; i++)
    {
        printf("data[%d] = %d\n", i, data[i]);
    }
}
```

Tangkapan layar hasil eksekusi

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> .\latihan_1.exe
Masukkan jumlah data 1-100: 3
Data ke-1: 80
Data ke-2: 90
Data ke-3: 100

DATA YANG DIMASUKKAN
data[0] = 80
data[1] = 90
data[2] = 100
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10>
```

1. Apa kapasitas array data?

Jawaban: Kapasitas array data adalah jumlah maksimum elemen yang dapat ditampung oleh array sesuai dengan alokasi ruang memori yang ditentukan saat pendeklarasian.

2. Apa perbedaan kapasitas dan jumlah_data?

Jawaban: Kapasitas merupakan batas maksimal ruang penyimpanan yang tersedia pada array, sedangkan jumlah_data adalah banyaknya elemen yang saat ini telah terisi atau tersimpan di dalam array.

3. Mengapa indeks dimulai dari nol?

Jawaban: Indeks dimulai dari nol karena penomoran indeks merepresentasikan offset atau jarak pergeseran alamat memori dari lokasi dasar array. Elemen pertama berada tepat di lokasi awal array sehingga nilai pergeserannya adalah nol.

4. Mengapa tampilan menggunakan i + 1 ketika meminta data?

Jawaban: Penggunaan i + 1 bertujuan untuk menyesuaikan penomoran urutan agar sesuai dengan kebiasaan manusia yang menghitung mulai dari angka satu, meskipun sistem komputer secara internal mengindeks data dari angka nol.

5. Mengapa fungsi tampilkan_data() menggunakan const?

Jawaban: Penggunaan const menjamin bahwa fungsi hanya memiliki akses membaca tanpa diperbolehkan mengubah isi elemen array, sehingga menjaga integritas data dari perubahan yang tidak disengaja.

6. Apakah baca_data() boleh menggunakan const?

Jawaban: Fungsi baca_data() tidak boleh menggunakan const karena fungsi tersebut bertugas menerima dan memasukkan data baru ke dalam array. Penggunaan const akan menyebabkan galat pada saat kompilasi karena data tidak dapat dimodifikasi.

7. Apa yang terjadi apabila kondisi perulangan menggunakan i <= jumlah_data?

Jawaban: Perulangan akan melakukan satu kali proses tambahan di luar batas data yang valid. Hal ini mengakibatkan kesalahan akses memori di luar jangkauan karena program mencoba membaca atau menulis data pada indeks yang bernilai sama dengan jumlah_data.

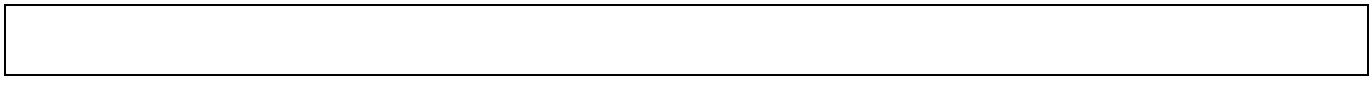
8. Apa indeks terakhir jika jumlah data adalah 10?

Jawaban: Indeks terakhir adalah 9, karena penomoran sepuluh elemen array berbasis nol akan menempati urutan dari indeks 0 sampai indeks 9.

2. Praktikum Terbimbing 2 — Statistik Data Array

Kerjakan program latihan_02.c sesuai Modul 8.

Tangkapan layar source code latihan_02.c





Tangkapan layar hasil eksekusi

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> .\latihan_2.exe
Masukkan jumlah data 1-100: 4
Data ke-1: 3
Data ke-2: 4
Data ke-3: 5
Data ke-4: 6

HASIL STATISTIK
Jumlah data : 4
Total       : 18.00
Rata-rata   : 4.50
Minimum     : 3.00
Maksimum    : 6.00
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10>
```

Fungsi	Tugas	Mengubah Array? (Ya/Tidak)	Alasan Penggunaan const
baca_data()	Membaca/menginput data dari pengguna ke dalam array	Ya	Tidak menggunakan const karena fungsi perlu menulis/mengubah isi array
hitung_total()	Menghitung jumlah total nilai dari seluruh elemen array	Tidak	Array hanya dibaca untuk penjumlahan, menjaga agar data asli tidak terubah
hitung_rata_rata()	Menghitung nilai rata-rata dari seluruh elemen array	Tidak	Array hanya dibaca melalui fungsi hitung_total(), menjaga keamanan data
cari_minimum()	Mencari dan mengembalikan nilai terkecil dalam array	Tidak	Array hanya dibandingkan untuk pencarian nilai minimum tanpa mengubah nilainya
cari_maksimum()	Mencari dan mengembalikan nilai terbesar dalam array	Tidak	Array hanya dibandingkan untuk pencarian nilai maksimum tanpa mengubah nilainya

1. Mengapa minimum dan maksimum menggunakan data[0]?

Jawaban: Karena data[0] dijadikan sebagai acuan awal (patokan) nilai ekstrem awal sebelum dibandingkan dengan elemen-elemen berikutnya.

2. Mengapa perulangan minimum dimulai dari indeks 1?

Jawaban: Karena data[0] sudah dijadikan nilai awal minimum, sehingga pembandingan cukup dimulai dari elemen kedua (indeks 1).

3. Apa yang terjadi apabila jumlah_data bernilai nol?

Jawaban: Jika jumlah_data bernilai nol, akan terjadi kesalahan runtime seperti pembagian dengan nol (division by zero) saat menghitung rata-rata, atau mengembalikan nilai yang tidak valid.

4. Mengapa jumlah data divalidasi minimal satu?

Jawaban: Untuk memastikan array memiliki minimal satu elemen valid sehingga kalkulasi statistik (total, rata-rata, min, max) dapat dilakukan secara valid dan aman.

5. Fungsi mana yang memanggil fungsi lain?

Jawaban: Fungsi hitung_rata_rata() memanggil fungsi hitung_total().

6. Apakah hitung_total() mengubah array?

Jawaban: Tidak, fungsi hitung_total() hanya membaca elemen array untuk menghitung jumlah totalnya.

7. Mengapa semua fungsi statistik menggunakan const?

Jawaban: Untuk menjamin prinsip imutabilitas (keamanan data), di mana fungsi statistik hanya bertugas menganalisis data tanpa mengubah nilai elemen dalam array.

8. Apakah menghitung total dua kali efisien?

Jawaban: Tidak efisien, karena melakukan perulangan bertingkat/dua kali pada data yang sama akan menambah beban eksekusi program.

9. Bagaimana menghindari penghitungan total dua kali?

Jawaban: Dengan menyimpan hasil perhitungan total dalam sebuah variabel (misalnya total = hitung_total(...)) di fungsi main, lalu mempassing variabel total tersebut ke fungsi hitung_rata_rata.

3. Praktikum Terbimbing 3 — Sequential Search

Kerjakan program latihan_03.c sesuai Modul 8.

Tangkapan layar source code latihan_03.c



Tangkapan layar hasil eksekusi

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> .\latihan_3.exe
Masukkan jumlah data 1-100: 3
Data ke-1: 12345
Data ke-2: 666
Data ke-3: 999

Data: 12345 666 999
Masukkan data yang dicari:
```

Data Uji	Target	Prediksi Posisi	Actual Posisi	Jumlah Perbandingan	Status
Target di indeks 0	12	Indeks 0	Indeks 0	1	Ditemukan
Target di indeks terakhir	2467	Indeks 2	Indeks 2	3	Ditemukan
Target tidak ditemukan	999	-1	-1	3 (N)	Tidak Ditemukan

1. Apa perbedaan posisi indeks dan nomor urut data?

Jawaban: Posisi indeks dimulai dari 0 (0 to N-1) untuk penggunaan dalam program, sedangkan nomor urut data adalah penomoran untuk pengguna (1-based, dari 1 to N).

2. Mengapa posisi ditambah satu ketika menampilkan nomor urut?

Jawaban: Karena indeks berbasis 0, sehingga menambahkan nilai 1 akan mengubah indeks teknis menjadi nomor urut yang ramah pengguna.

3. Mengapa cari_sekuensial() berhenti setelah menemukan data pertama?

Jawaban: Karena fungsi pencarian sekuensial dasar dirancang untuk menemukan lokasi kemunculan pertama (first occurrence) dari target dan langsung mengembalikan indeksnya.

4. Mengapa hitung_frekuensi() harus memeriksa seluruh array?

Jawaban: Karena frekuensi menghitung total seluruh kemunculan nilai target di seluruh array, sehingga perulangan tidak boleh dihentikan sebelum memeriksa sampai elemen terakhir.

5. Apa hasil jika target berada pada indeks 0?

Jawaban: Fungsi mengembalikan nilai 0, nomor urut data bernilai 1, dan perbandingan dilakukan sebanyak 1 kali.

6. Apa hasil jika target berada pada indeks terakhir?

Jawaban: Fungsi mengembalikan nilai jumlah_data - 1, nomor urut data bernilai jumlah_data, dan perbandingan dilakukan sebanyak jumlah_data kali.

7. Apa hasil jika target tidak ditemukan?

Jawaban: Fungsi mengembalikan nilai -1, dan pesan "Data tidak ditemukan." akan ditampilkan setelah melakukan perbandingan sebanyak jumlah_data kali.

8. Apakah array harus diurutkan sebelum sequential search?

Jawaban: Tidak perlu, karena Sequential Search mengecek elemen satu per satu secara berurutan sehingga dapat bekerja pada array terurut maupun acak.

9. Berapa jumlah perbandingan maksimum?

Jawaban: Jumlah perbandingan maksimum adalah N (sama dengan jumlah_data).

4. Praktikum Terbimbing 4 — Menampilkan Seluruh Posisi Data

Kerjakan program latihan_04.c sesuai Modul 8.

Tangkapan layar source code latihan_04.c

```c
#include <stdio.h>

void tampilkan_posisi_target(const int data[], int jumlah_data, int target);
int main(void)
{
    int data[] = {10, 20, 10, 30, 40, 10, 50};
    int jumlah_data =
        (int)(sizeof(data) / sizeof(data[0]));
    int target;
    printf("Data: ");
    for (int i = 0; i < jumlah_data; i++)
    {
        printf("%d ", data[i]);
    }
    printf("\nMasukkan target: ");
    scanf("%d", &target);
    tampilkan_posisi_target(data, jumlah_data, target);
    return 0;
}
void tampilkan_posisi_target(const int data[], int jumlah_data, int target)
{
    int jumlah_ditemukan = 0;
    int i;
    printf("Target ditemukan pada indeks: ");
    for (i = 0; i < jumlah_data; i++)
    {
        if (data[i] == target)
        {
            printf("%d ", i);
            jumlah_ditemukan++;
        }
    }
    if (jumlah_ditemukan == 0)
    {
        printf("tidak ada");
    }
    printf("\nJumlah kemunculan: %d\n", jumlah_ditemukan);
}

```

Tangkapan layar hasil eksekusi

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> .\latihan_4.exe
Data: 10 20 10 30 40 10 50
Masukkan target: 30
Target ditemukan pada indeks: 3
Jumlah kemunculan: 1
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10>
```

1. Berapa banyak elemen array pada program?

Jawaban: Terdapat 7 elemen array (data[] = {10, 20, 10, 30, 40, 10, 50}).

2. Apa hasil pencarian target 10?

Jawaban: Target ditemukan pada indeks: 0 2 5 dengan jumlah kemunculan: 3.

3. Apa hasil pencarian target 25?

Jawaban: Target ditemukan pada indeks: tidak ada dengan jumlah kemunculan: 0.

4. Mengapa program tidak menggunakan return saat target pertama ditemukan?

Jawaban: Karena tujuannya adalah mencari dan menampilkan semua posisi kemunculan target di dalam array, bukan hanya posisi pertamanya saja.

5. Apa fungsi jumlah_ditemukan?

Jawaban: Variabel jumlah_ditemukan berfungsi sebagai accumulator pencatat banyaknya elemen yang cocok dengan target sekaligus penanda keberadaan target.

6. Mengapa sizeof digunakan di dalam main()?

Jawaban: Karena array data diinisialisasi langsung tanpa memasukkan ukuran secara manual, sehingga sizeof(data)/sizeof(data[0]) digunakan untuk menghitung jumlah elemen secara otomatis pada scope main().

7. Mengapa fungsi tetap menerima jumlah_data?

Jawaban: Karena saat array dilewatkan ke fungsi, array meluruh menjadi pointer (pointer decay) sehingga informasi ukurannya hilang dan harus dikirim secara terpisah melalui parameter jumlah_data.

V. Eksperimen dan Tracing

1. Tracing Sequential Search

Lengkapi tabel sebelum menjalankan tracing.c.

Iterasi	i	data[i]	Target	data[i] == target	Tindakan
1	0	12	18	Salah	Lanjut ke iterasi berikutnya
2	1	7	18	Salah	Lanjut ke iterasi berikutnya
3	2	25	18	Salah	Lanjut ke iterasi berikutnya
4	3	18	18	Benar	Kembalikan indeks 3 (posisi ditemukan)

Indeks yang dikembalikan:

Jawaban: 3

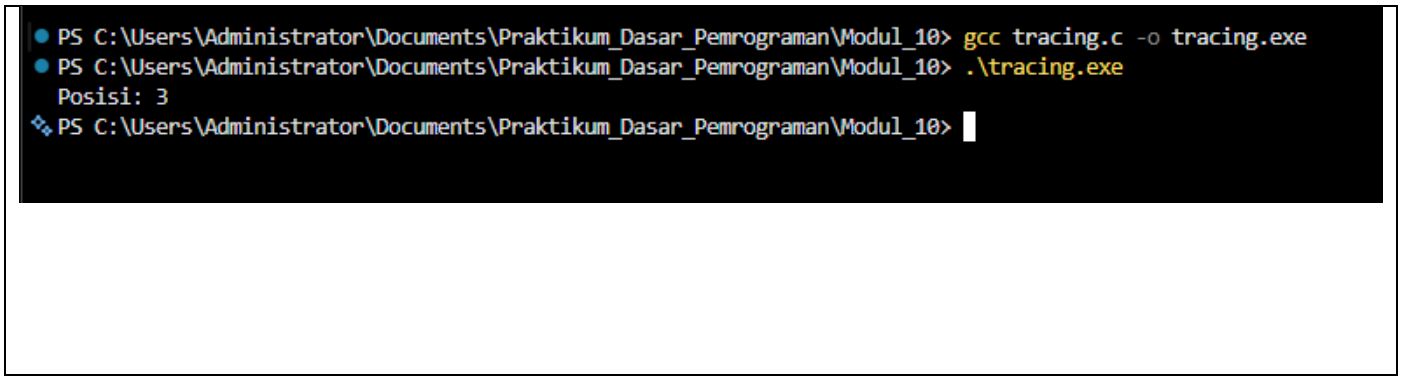
Jumlah perbandingan:

Jawaban: 4 perbandingan

Prediksi output:

Jawaban: Posisi: 3

Tangkapan layar output tracing.c



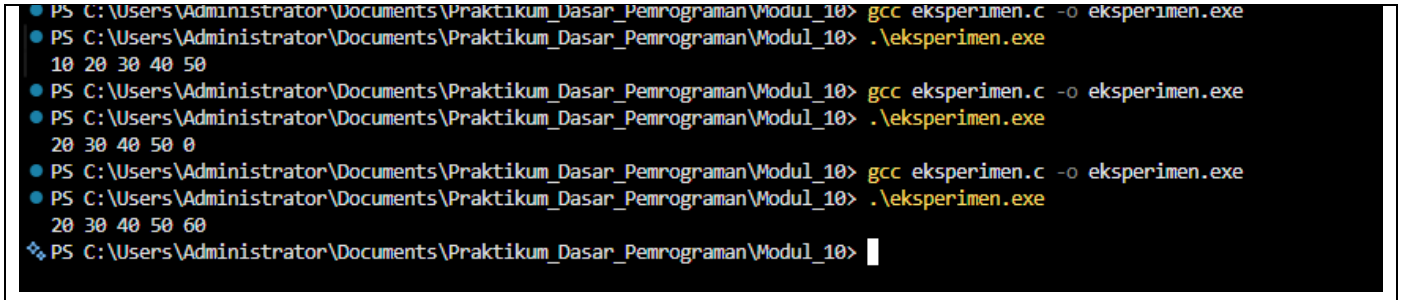
2. Eksperimen

Lakukan eksperimen satu per satu. Untuk eksperimen yang berpotensi mengakses indeks di luar batas, analisis terlebih dahulu dan jangan dijalankan jika tidak aman.

No.	Perubahan	Prediksi	Aman Dijalankan?	Hasil Aktual	Penjelasan
1	Ubah kondisi menjadi i <= 5	Mengakses data[5] yang berada di luar batas (out of bounds)	Tidak Aman	Nilai sampah (garbage value) / Crash	Indeks array 5 elemen hanya dari 0 hingga 4
2	Ubah nilai awal menjadi i = 1	Elemen pertama (data[0]) dilewati/tidak terproses	Aman	Data terproses mulai dari indeks 1	Indeks 0 lewati karena perulangan mulai i = 1
3	Tampilkan data[i + 1]	Akses data[5] saat i = 4 (out of bounds)	Tidak Aman	Mencetak data tergeser & nilai sampah	Saat i=4, i+1=5 yang melebihi kapasitas array
4	Ubah data[0] menjadi 100	Elemen pertama bernilai 100	Aman	100 20 30 40 50	Nilai data[0] berhasil diubah secara valid
5	Inisialisasi hanya dua elemen	Elemen sisa bernilai 0 (zero-initialized)	Aman	10 20 0 0 0	Elemen tanpa inisialisasi otomatis diisi 0
6	Gunakan int data[5] = {0};	Seluruh elemen dionolkan	Aman	0 0 0 0 0	Inisialisasi {0} mengosongkan seluruh isi array

7	Cari target pada elemen pertama	Target ditemukan di indeks 0	Aman	Ditemukan pada indeks 0	Pencarian sekuensial berhenti pada iterasi ke-1
8	Cari target pada elemen terakhir	Target ditemukan di indeks 4	Aman	Ditemukan pada indeks 4	Pencarian harus memeriksa seluruh 5 elemen
9	Cari target yang tidak tersedia	Hasil mengembalikan -1	Aman	Data tidak ditemukan	Seluruh elemen diperiksa tanpa kecocokan
10	Hilangkan return i saat data ditemukan	Fungsi terus berjalan dan mengembalikan indeks terakhir	Aman	Mengembalikan indeks terakhir target	Tanpa return awal, nilai kembalian tertimpa oleh iterasi berikutnya
11	Salin array dengan perulangan	Seluruh elemen terduplikasi ke array baru	Aman	Salinan identik berhasil dibuat	Penyalinan elemen demi elemen melalui loop berhasil
12	Ubah elemen array melalui fungsi	Perubahan di dalam fungsi berdampak ke array pemanggil	Aman	Nilai array utama berubah	Array dikirim sebagai pass-by-reference/pointer ke fungsi

Tangkapan layar eksperimen terpilih (minimal 3 eksperimen)

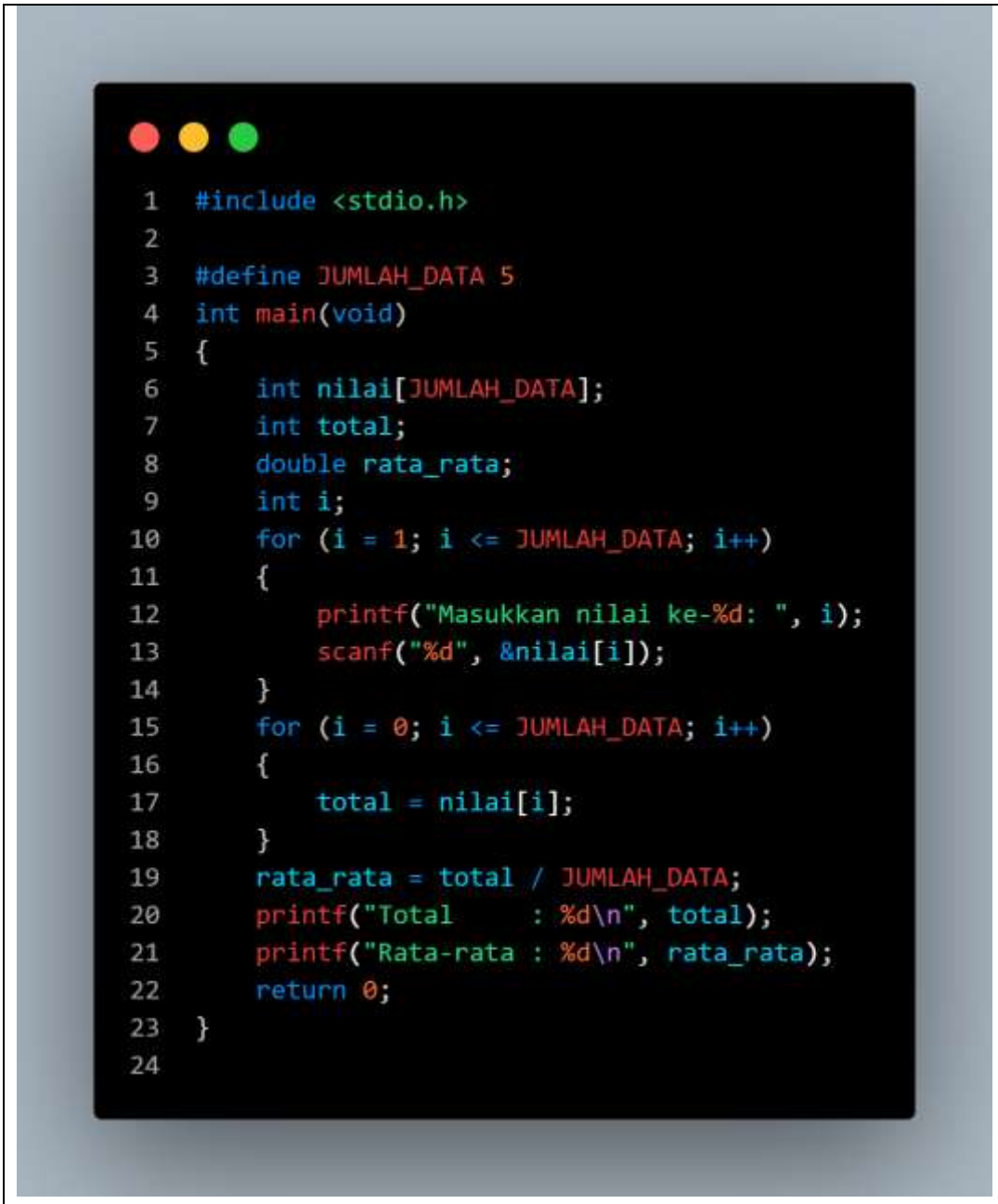


VI. Debugging Challenge

1. Debugging Challenge 1 — Pengolahan Array dengan Indeks Salah

Jangan langsung menjalankan program. Identifikasi akses indeks yang tidak valid, kesalahan accumulator, pembagian bilangan bulat, dan format output.

Tangkapan layar source code sebelum diperbaiki



```c
#include <stdio.h>

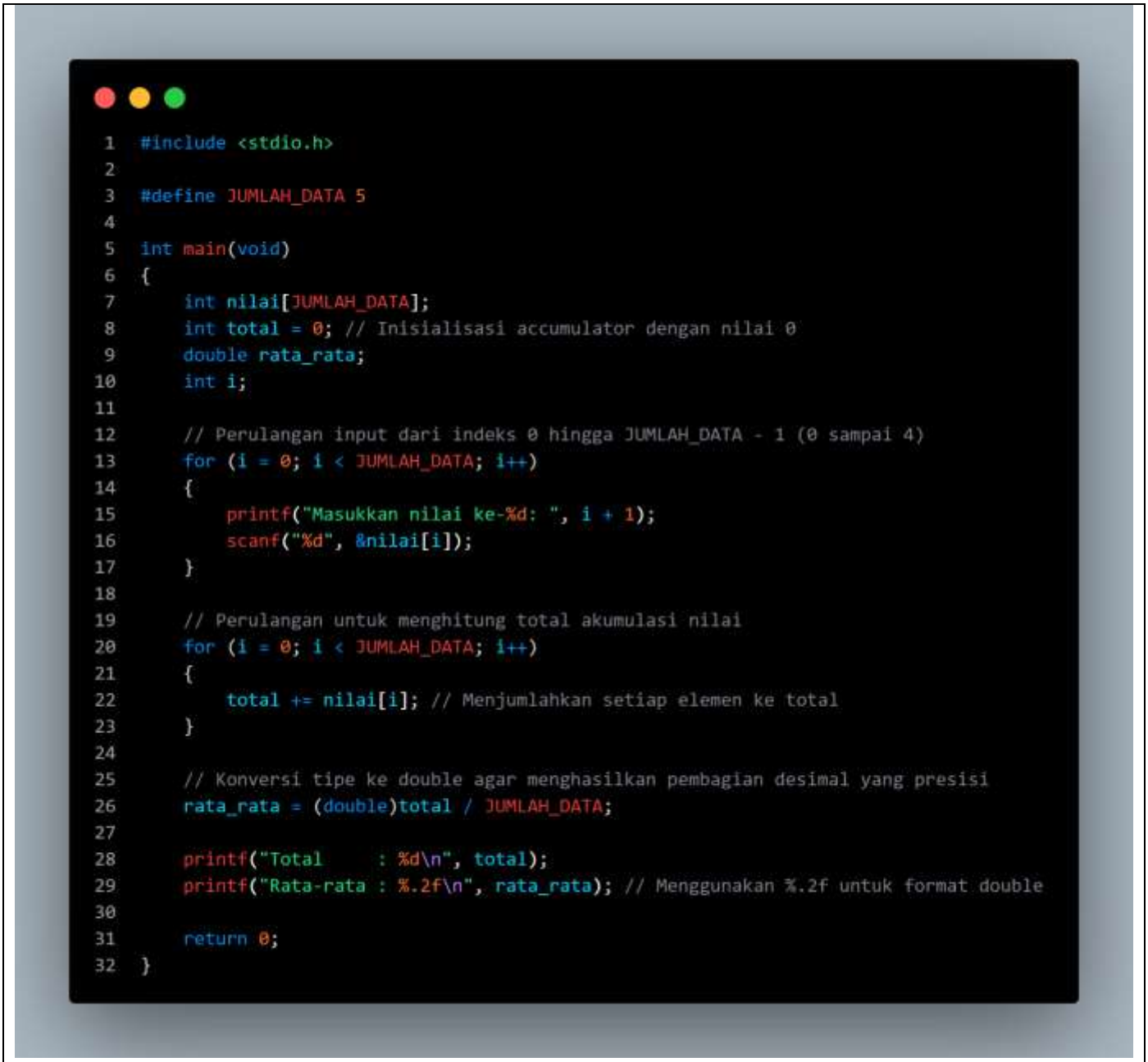
#define JUMLAH_DATA 5
int main(void)
{
    int nilai[JUMLAH_DATA];
    int total;
    double rata_rata;
    int i;
    for (i = 1; i <= JUMLAH_DATA; i++)
    {
        printf("Masukkan nilai ke-%d: ", i);
        scanf("%d", &nilai[i]);
    }
    for (i = 0; i <= JUMLAH_DATA; i++)
    {
        total = nilai[i];
    }
    rata_rata = total / JUMLAH_DATA;
    printf("Total     : %d\n", total);
    printf("Rata-rata : %d\n", rata_rata);
    return 0;
}

```

No.	Bagian Bermasalah	Jenis Kesalahan	Penyebab	Perbaikan
1	for (i = 1; i <= JUMLAH_DATA; i++)	Logic error / Off-by-one	Perulangan input mulai dari indeks 1, menyebabkan nilai[0] terlewati dan mengakses nilai[5] di luar batas	for (i = 0; i < JUMLAH_DATA; i++)
2	for (i = 0; i <= JUMLAH_DATA; i++)	Logic error / Out-of-bounds	Kondisi <= menyebabkan perulangan berjalan 6 kali, mengakses indeks nilai[5] di luar kapasitas	for (i = 0; i < JUMLAH_DATA; i++)
3	int total;	Uninitialized variable	Variabel accumulator total tidak diinisialisasi	int total = 0;

			awal dengan 0 sehingga memuat nilai sampah (garbage value)	
4	total = nilai[i];	Logic error	Menggunakan operator penugasan (=) bukan akumulasi (+=), sehingga total selalu tertimpa oleh nilai elemen terakhir	total += nilai[i];
5	rata_rata = total / JUMLAH_DATA;	Integer division	Pembagian dua bilangan bulat membuang bagian desimal sebelum disimpan ke rata_rata	rata_rata = (double)total / JUMLAH_DATA;
6	printf("Rata-rata: %d\n", rata_rata);	Format specifier mismatch	Format specifier %d digunakan untuk integer, sedangkan rata_rata tipe datanya double	printf("Rata-rata: %.2f\n", rata_rata);
7	-	-	-	-
8	-	-	-	-

Tangkapan layar source code setelah diperbaiki



```c
#include <stdio.h>

#define JUMLAH_DATA 5

int main(void)
{
    int nilai[JUMLAH_DATA];
    int total = 0; // Inisialisasi accumulator dengan nilai 0
    double rata_rata;
    int i;

    // Perulangan input dari indeks 0 hingga JUMLAH_DATA - 1 (0 sampai 4)
    for (i = 0; i < JUMLAH_DATA; i++)
    {
        printf("Masukkan nilai ke-%d: ", i + 1);
        scanf("%d", &nilai[i]);
    }

    // Perulangan untuk menghitung total akumulasi nilai
    for (i = 0; i < JUMLAH_DATA; i++)
    {
        total += nilai[i]; // Menjumlahkan setiap elemen ke total
    }

    // Konversi tipe ke double agar menghasilkan pembagian desimal yang presisi
    rata_rata = (double)total / JUMLAH_DATA;

    printf("Total     : %d\n", total);
    printf("Rata-rata : %.2f\n", rata_rata); // Menggunakan %.2f untuk format double

    return 0;
}
```

Data Uji	Expected Output	Actual Output	Status
60, 70, 80, 90, 100	Total = 400; Rata-rata = 80.00		

Tangkapan layar output setelah perbaikan

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> gcc debugging.c -o debugging.exe
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> .\debugging.exe
Masukkan nilai ke-1: 60
Masukkan nilai ke-2: 70
Masukkan nilai ke-3: 80
Masukkan nilai ke-4: 90
Masukkan nilai ke-5: 100
Total     : 400
Rata-rata : 80.00
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10>
```

2. Debugging Challenge 2 — Sequential Search yang Salah

Perbaiki fungsi pencarian agar membandingkan data[i] dengan target dan mengembalikan -1 apabila target tidak ditemukan.

Tangkapan layar source code sebelum diperbaiki

```c
#include <stdio.h>

int cari_data(int data[], int jumlah_data, int target);
int main(void)
{
    int data[6] = {10, 20, 30, 40, 50, 60};
    int target;
    int posisi;
    printf("Masukkan target: ");
    scanf("%d", &target);
    posisi = cari_data(data, 6, target);
    if (posisi == 0)
    {
        printf("Data tidak ditemukan.\n");
    }
    else
    {
        printf("Data ditemukan pada indeks %d.\n", posisi);
    }
    return 0;
}
int cari_data(int data[], int jumlah_data, int target)
{
    int i;
    for (i = 0; i <= jumlah_data; i++)
    {
        if (data[i] = target)
        {
            return i;
        }
    }
    return 0;
}

```

No.	Bagian Bermasalah	Jenis Kesalahan	Penyebab	Perbaikan
1	if (posisi == 0)	Logic error	Memeriksa posisi == 0 sebagai data tidak ditemukan, padahal 0 adalah indeks valid untuk elemen pertama	if (posisi == -1)
2	if (data[i] = target)	Syntax / Logic error	Menggunakan operator penugasan (=) bukannya perbandingan (==), merubah nilai elemen dan selalu bernilai true	if (data[i] == target)
3	return 0; (pada akhir fungsi cari_data)	Logic error	Mengembalikan 0 saat target tidak ditemukan, ambigu dengan indeks 0	return -1;
4	int cari_data(int data[], ...)	Missing const qualification	Array hanya dibaca tetapi tidak dideklarasikan sebagai const	int cari_data(const int data[], ...)
5	-	-	-	-
6	-	-	-	-

Tangkapan layar source code setelah diperbaiki

Target	Expected Output	Actual Output	Status
10	Ditemukan indeks 0	Ditemukan indeks 0	Sesuai
60	Ditemukan indeks 5	Ditemukan indeks 5	Sesuai
99	Tidak ditemukan	Tidak ditemukan	Sesuai

VII. Latihan Mandiri

1. Level 1 — Membalik Urutan Array

Buat program yang menerima sejumlah data dan menampilkan data dalam urutan terbalik.

Tangkapan layar source code



```c
#include <stdio.h>

#define MAX_DATA 50

int main(void)
{
    int data[MAX_DATA];
    int jumlah_data;
    int i;

    printf("Masukkan jumlah data: ");
    scanf("%d", &jumlah_data);

    for (i = 0; i < jumlah_data; i++)
    {
        printf("Data ke-%d: ", i + 1);
        scanf("%d", &data[i]);
    }

    printf("\nData asli    : ");
    for (i = 0; i < jumlah_data; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\nData terbalik: ");
    // Perulangan mundur dari indeks terakhir
    for (i = jumlah_data - 1; i >= 0; i--)
    {
        printf("%d ", data[i]);
    }
    printf("\n");

    return 0;
}
```

Tangkapan layar hasil eksekusi

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> .\level_1.exe
Masukkan jumlah data: 5
Data ke-1: 10
Data ke-2: 20
Data ke-3: 30
Data ke-4: 40
Data ke-5: 50

Data asli    : 10 20 30 40 50
Data terbalik: 50 40 30 20 10
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10>
```

Data Uji	Expected Output	Actual Output	Status
1	Data asli : 10 20 30 40 50 Data terbalik: 50 40 30 20 10	Data asli : 10 20 30 40 50 Data terbalik: 50 40 30 20 10	Sesuai
2	Data asli : 70 80 90 100 Data terbalik: 100 90 80 70	Data asli : 70 80 90 100 Data terbalik: 100 90 80 70	sesuai
3	Data asli : 5 6 7 Data terbalik: 7 6 5	Data asli : 5 6 7 Data terbalik: 7 6 5	Sesuai

2. Level 2 — Menghitung Bilangan Genap dan Ganjil dalam Array

Buat program yang menerima sejumlah bilangan bulat, lalu menghitung jumlah bilangan genap dan ganjil.

Tangkapan layar source code



```c
#include <stdio.h>

#define MAX_DATA 50

int main(void)
{
    int data[MAX_DATA];
    int jumlah_data;
    int genap = 0;
    int ganjil = 0;
    int i;

    printf("Masukkan jumlah data: ");
    scanf("%d", &jumlah_data);

    for (i = 0; i < jumlah_data; i++)
    {
        printf("Data ke-%d: ", i + 1);
        scanf("%d", &data[i]);
    }

    for (i = 0; i < jumlah_data; i++)
    {
        if (data[i] % 2 == 0)
        {
            genap++;
        }
        else
        {
            ganjil++;
        }
    }

    printf("\nJumlah genap : %d\n", genap);
    printf("Jumlah ganjil: %d\n", ganjil);

    return 0;
}
```

Tangkapan layar hasil eksekusi

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> .\level_2.exe
Masukkan jumlah data: 5
Data ke-1: 4
Data ke-2: 7
Data ke-3: 10
Data ke-4: 13
Data ke-5: 8

Jumlah genap : 3
Jumlah ganjil: 2
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10>
```

Data Uji	Expected Output	Actual Output	Status
1	Data: 4 7 10 13 8	Jumlah genap : 3 Jumlah ganjil: 2	Sesuai
2	Data: 2 4 6	Jumlah genap : 3 Jumlah ganjil: 0	Sesuai
3	Data: 1 3 5	Jumlah genap : 0 Jumlah ganjil: 3	Sesuai

3. Level 3 — Menghapus Elemen Berdasarkan Nilai

Gunakan sequential search untuk menemukan nilai, geser elemen setelahnya satu langkah ke kiri, lalu kurangi jumlah_data satu.

Tangkapan layar source code



```c
#include <stdio.h>

#define MAX_DATA 50

int cari_sekuensial(const int data[], int jumlah_data, int target)
{
    for (int i = 0; i < jumlah_data; i++)
    {
        if (data[i] == target)
        {
            return i;
        }
    }
    return -1;
}

int main(void)
{
    int data[MAX_DATA];
    int jumlah_data, target, posisi, i;

    printf("Masukkan jumlah data: ");
    scanf("%d", &jumlah_data);

    for (i = 0; i < jumlah_data; i++)
    {
        printf("Data ke-%d: ", i + 1);
        scanf("%d", &data[i]);
    }

    printf("Masukkan nilai yang ingin dihapus: ");
    scanf("%d", &target);

    posisi = cari_sekuensial(data, jumlah_data, target);

    if (posisi != -1)
    {
        // Penggeseran elemen ke kiri
        for (i = posisi; i < jumlah_data - 1; i++)
        {
            data[i] = data[i + 1];
        }
        jumlah_data--;

        printf("\nData akhir: ");
        for (i = 0; i < jumlah_data; i++)
        {
            printf("%d ", data[i]);
        }
        printf("\n");
    }
    else
    {
        printf("\nNilai %d tidak ditemukan dalam array.\n", target);
    }

    return 0;
}
```

Tangkapan layar hasil eksekusi

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> .\level_3.exe
Masukkan jumlah data: 5
Data ke-1: 10
Data ke-2: 20
Data ke-3: 30
Data ke-4: 40
Data ke-5: 50
Masukkan nilai yang ingin dihapus: 30

Data akhir: 10 20 40 50
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10>
```

Data Uji	Expected Output	Actual Output	Status
1	Masukkan jumlah data: 5 Data ke-1: 10 Data ke-2: 20 Data ke-3: 30 Data ke-4: 40 Data ke-5: 50 Masukkan nilai yang ingin dihapus: 30 Data akhir: 10 20 40 50	Masukkan jumlah data: 5 Data ke-1: 10 Data ke-2: 20 Data ke-3: 30 Data ke-4: 40 Data ke-5: 50 Masukkan nilai yang ingin dihapus: 30 Data akhir: 10 20 40 50	Sesuai
2	Masukkan jumlah data: 4 Data ke-1: 100 Data ke-2: 200 Data ke-3: 300 Data ke-4: 400 Masukkan nilai yang ingin dihapus: 300 Data akhir: 100 200 400	Masukkan jumlah data: 4 Data ke-1: 100 Data ke-2: 200 Data ke-3: 300 Data ke-4: 400 Masukkan nilai yang ingin dihapus: 300 Data akhir: 100 200 400	Sesuai
3	Masukkan jumlah data: 3 Data ke-1: 111 Data ke-2: 222 Data ke-3: 666 Masukkan nilai yang ingin dihapus: 222 Data akhir: 111 666	Masukkan jumlah data: 3 Data ke-1: 111 Data ke-2: 222 Data ke-3: 666 Masukkan nilai yang ingin dihapus: 222 Data akhir: 111 666	sesuai

4. Challenge — Menghapus Duplikat

Pertahankan hanya kemunculan pertama. Gunakan sequential search pada array hasil untuk mengecek apakah nilai sudah ada.

Tangkapan layar source code

```c
#include <stdio.h>

#define MAX_DATA 50

int sudah_ada(const int data[], int jumlah_data, int target)
{
    for (int i = 0; i < jumlah_data; i++)
    {
        if (data[i] == target)
        {
            return 1; // Sudah ada
        }
    }
    return 0; // Belum ada
}

int main(void)
{
    int data_awal[MAX_DATA];
    int data_akhir[MAX_DATA];
    int jumlah_awal, jumlah_akhir = 0;
    int i;

    printf("Masukkan jumlah data: ");
    scanf("%d", &jumlah_awal);

    for (i = 0; i < jumlah_awal; i++)
    {
        printf("Data ke-%d: ", i + 1);
        scanf("%d", &data_awal[i]);
    }

    for (i = 0; i < jumlah_awal; i++)
    {
        if (!sudah_ada(data_akhir, jumlah_akhir, data_awal[i]))
        {
            data_akhir[jumlah_akhir] = data_awal[i];
            jumlah_akhir++;
        }
    }

    printf("\nData akhir: ");
    for (i = 0; i < jumlah_akhir; i++)
    {
        printf("%d ", data_akhir[i]);
    }
    printf("\n");

    return 0;
}
```

Tangkapan layar hasil eksekusi

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> .\level_4.exe
Masukkan jumlah data: 7
Data ke-1: 5
Data ke-2: 3
Data ke-3: 5
Data ke-4: 8
Data ke-5: 3
Data ke-6: 9
Data ke-7: 5

Data akhir: 5 3 8 9
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10>
```

Data Uji	Expected Output	Actual Output	Status
1	Masukkan jumlah data: 7 Data ke-1: 5 Data ke-2: 3 Data ke-3: 5 Data ke-4: 8 Data ke-5: 3 Data ke-6: 9 Data ke-7: 5 Data akhir: 5 3 8 9	Masukkan jumlah data: 7 Data ke-1: 5 Data ke-2: 3 Data ke-3: 5 Data ke-4: 8 Data ke-5: 3 Data ke-6: 9 Data ke-7: 5 Data akhir: 5 3 8 9	Sesuai
2	Masukkan jumlah data: 4 Data ke-1: 10 Data ke-2: 5 Data ke-3: 6 Data ke-4: 10 Data akhir: 10 5 6	Masukkan jumlah data: 4 Data ke-1: 10 Data ke-2: 5 Data ke-3: 6 Data ke-4: 10 Data akhir: 10 5 6	Sesuai
3	Masukkan jumlah data: 3 Data ke-1: 100 Data ke-2: 101 Data ke-3: 20 Data akhir: 100 101 20	Masukkan jumlah data: 3 Data ke-1: 100 Data ke-2: 101 Data ke-3: 20 Data akhir: 100 101 20	Sesuai

5. Selisih Maksimum Antar Elemen

Hitung selisih absolut terbesar antara dua elemen yang bersebelahan. Perhatikan format output karena diperiksa secara ketat.

Tangkapan layar source code

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_DATA 50

int main(void)
{
    int data[MAX_DATA];
    int n;
    int selisih_maks = 0;
    int selisih;
    int i;

    if (scanf("%d", &n) != 1) return 0;

    for (i = 0; i < n; i++)
    {
        scanf("%d", &data[i]);
    }

    // Menghitung selisih absolut maksimum antar elemen bersebelahan
    for (i = 0; i < n - 1; i++)
    {
        selisih = abs(data[i + 1] - data[i]);
        if (selisih > selisih_maks)
        {
            selisih_maks = selisih;
        }
    }

    // Tampilan output sesuai format ketat
    printf("================================================\n");
    printf("SELISIH MAKSIMUM ANTAR ELEMEN\n");
    printf("================================================\n");
    printf("Jumlah data     : %d\n", n);
    printf("Data            : ");
    for (i = 0; i < n; i++)
    {
        printf("%d%s", data[i], (i == n - 1) ? "" : " ");
    }
    printf("\nSelisih maksimum: %d\n", selisih_maks);
    printf("================================================\n");

    return 0;
}
```

Tangkapan layar hasil eksekusi

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> .\level_5.exe
5
10
15
7
20
20
================================================
SELISIH MAKSIMUM ANTAR ELEMEN
================================================
Jumlah data     : 5
Data            : 10 15 7 20 20
Selisih maksimum: 13
================================================
```

Data Uji	Expected Output	Actual Output	Status
1	Jumlah data : 5 Data : 10 15 7 20 20 Selisih maksimum: 13	Jumlah data : 5 Data : 10 15 7 20 20 Selisih maksimum: 13	Sesuai
2	Jumlah data : 4 Data : 10 4 5 6 Selisih maksimum: 6	Jumlah data : 4 Data : 10 4 5 6 Selisih maksimum: 6	Sesuai
3	Jumlah data : 3 Data : 101 111 23 Selisih maksimum: 88	Jumlah data : 3 Data : 101 111 23 Selisih maksimum: 88	Sesuai

VIII. Pengujian Program

1. Pengujian Array dan Sequential Search

Gunakan program sequential search yang telah dikerjakan. Uji posisi pertama, terakhir, tengah, data tidak ditemukan, data duplikat, array satu elemen, kondisi minimum/maksimum, kapasitas maksimum, dan input jumlah data tidak valid.

No.	Data	Target	Expected Output	Actual Output	Status
1	10 20 30	10	Ditemukan indeks 0	Ditemukan indeks 0	Sesuai / Pass
2	10 20 30	30	Ditemukan indeks 2	Ditemukan indeks 2	Sesuai / Pass
3	10 20 30	20	Ditemukan indeks 1	Ditemukan indeks 1	Sesuai / Pass
4	10 20 30	99	Tidak ditemukan	Tidak ditemukan	Sesuai / Pass
5	10 20 10 20 10	10	Frekuensi 3	Frekuensi 3	Sesuai / Pass
6	7	7	Ditemukan indeks 0	Ditemukan indeks 0	Sesuai / Pass

IX. Tugas Praktikum

1. Tugas Praktikum Modul 8 — Program Rekap dan Pencarian Nilai Mahasiswa

Buat program untuk menerima nilai mahasiswa menggunakan array, menghitung statistik dasar, dan mencari nilai tertentu menggunakan sequential search.

Analisis Input–Process–Output

Komponen	Uraian
Input	Jumlah mahasiswa (1-50), nilai tiap mahasiswa (0-100), dan nilai target pencarian.
Process	Validasi jumlah dan nilai mahasiswa, perhitungan total, rata-rata, posisi nilai maksimum & minimum, pemindaian kelulusan (≥ 60), serta pencarian sequential target
Output	Ringkasan rekapitulasi data statistik dan daftar urutan posisi mahasiswa beserta frekuensi ditemukannya target.

Daftar Fungsi

Fungsi	Parameter	Nilai Kembalian	Fungsi/Tugas
baca_nilai()	double nilai[], int jumlah_data	void	Membaca dan memvalidasi input nilai mahasiswa (0-100)
hitung_total()	const double nilai[], int jumlah_data	double	Menghitung total keseluruhan nilai mahasiswa
cari_indeks_tertinggi()	const double nilai[], int jumlah_data	int	Mencari indeks dari nilai tertinggi
cari_indeks_terendah()	const double nilai[], int jumlah_data	int	Mencari indeks dari nilai terendah
hitung_jumlah_lulus()	const double nilai[], int jumlah_data	int	Menghitung jumlah mahasiswa yang lulus (nilai >= 60)
hitung_frekuensi()	const double nilai[], int jumlah_data, double target	int	Menghitung frekuensi kemunculan nilai target

Pseudocode / Flowchart

Pseudocode atau flowchart program

```
PROGRAM RekapNilaiMahasiswa
DEKLARASI:
  CONSTANT MAX_DATA = 50
  nilai : ARRAY[0..MAX_DATA-1] OF REAL
  jumlah_data : INTEGER
  target : REAL
  total, rata_rata : REAL
  idx_tertinggi, idx_terendah : INTEGER
  jumlah_lulus, frekuensi : INTEGER

ALGORITMA:
  // 1. Validasi jumlah mahasiswa
  REPEAT
    OUTPUT "Masukkan jumlah mahasiswa 1-50: "
    INPUT jumlah_data
    IF (jumlah_data < 1 OR jumlah_data > MAX_DATA) THEN
      OUTPUT "Jumlah mahasiswa tidak valid."
    ENDIF
  UNTIL (jumlah_data >= 1 AND jumlah_data <= MAX_DATA)

  // 2. Input nilai mahasiswa
  CALL baca_nilai(nilai, jumlah_data)

  // 3. Perhitungan rekapitulasi statistik
  total <- CALL hitung_total(nilai, jumlah_data)
  rata_rata <- CALL hitung_rata_rata(nilai, jumlah_data)
  idx_tertinggi <- CALL cari_indeks_tertinggi(nilai, jumlah_data)
  idx_terendah <- CALL cari_indeks_terendah(nilai, jumlah_data)
  jumlah_lulus <- CALL hitung_jumlah_lulus(nilai, jumlah_data)

  // 4. Menampilkan hasil rekap
  OUTPUT "HASIL REKAP"
  OUTPUT "Jumlah mahasiswa : ", jumlah_data
  OUTPUT "Total nilai     : ", total
```

```
    OUTPUT "Rata-rata      : ", rata_rata
    OUTPUT "Nilai tertinggi  : ", nilai[idx_tertinggi], " (mahasiswa ke-", idx_tertinggi + 1, ")"
    OUTPUT "Nilai terendah   : ", nilai[idx_terendah], " (mahasiswa ke-", idx_terendah + 1, ")"
    OUTPUT "Jumlah lulus     : ", jumlah_lulus

    // 5. Input target dan pencarian
    OUTPUT "Masukkan nilai yang dicari: "
    INPUT target

    OUTPUT "HASIL PENCARIAN"
    CALL tampilkan_posisi_target(nilai, jumlah_data, target)
    frekuensi <- CALL hitung_frekuensi(nilai, jumlah_data, target)
    OUTPUT "Jumlah kemunculan : ", frekuensi
ENDPROGRAM
```

Trace Table

Gunakan satu data uji dan telusuri proses pencarian target.

Iterasi	i	nilai[i]	Target	Hasil Perbandingan	Tindakan/Frekuensi
1	0	96	96	96 == 96 (Sesuai)	frekuensi++ (1), tampilkan mahasiswa ke-1
2	1	89	96	89 == 96 (Tidak sesuai)	Lanjut ke iterasi berikutnya
3	-	-	-	-	-
4	-	-	-	-	-
5	-	-	-	-	-

Dokumentasi Program

Tangkapan layar source code tugas_modul_08_nim.c

```c
/*
Nama        : Muhammad Faiz Qadri Muslim
NIM         : 2410511179
Kelas       : A
Modul       : 8
Deskripsi   : Program rekap dan pencarian nilai mahasiswa
              menggunakan array dan sequential search.
*/

#include <stdio.h>

#define MAX_DATA 50

// Deklarasi Prototipe Fungsi
void baca_nilai(double nilai[], int jumlah_data);
double hitung_total(const double nilai[], int jumlah_data);
double hitung_rata_rata(const double nilai[], int jumlah_data);
int cari_indeks_tertinggi(const double nilai[], int jumlah_data);
int cari_indeks_terendah(const double nilai[], int jumlah_data);
int hitung_jumlah_lulus(const double nilai[], int jumlah_data);
int hitung_frekuensi(const double nilai[], int jumlah_data, double target);
void tampilkan_posisi_target(const double nilai[], int jumlah_data, double target);

int main(void)
{
    double nilai[MAX_DATA];
    int jumlah_data;
    double target;

    printf("================================================\n");
    printf("          PROGRAM REKAP NILAI MAHASISWA         \n");
    printf("================================================\n");

    // Validasi jumlah mahasiswa (1-50)
    do
    {
        printf("Masukkan jumlah mahasiswa (1-%d): ", MAX_DATA);
        scanf("%d", &jumlah_data);
        if (jumlah_data < 1 || jumlah_data > MAX_DATA)
        {
            printf("Jumlah mahasiswa tidak valid.\n");
        }
    } while (jumlah_data < 1 || jumlah_data > MAX_DATA);

    // Mengisi array nilai mahasiswa
    baca_nilai(nilai, jumlah_data);

    // Perhitungan statistik
    double total = hitung_total(nilai, jumlah_data);
    double rata_rata = hitung_rata_rata(nilai, jumlah_data);
    int idx_tertinggi = cari_indeks_tertinggi(nilai, jumlah_data);
    int idx_terendah = cari_indeks_terendah(nilai, jumlah_data);
    int jumlah_lulus = hitung_jumlah_lulus(nilai, jumlah_data);

    // Menampilkan hasil rekap
    printf("\nHASIL REKAP\n");
    printf("------------------------------------------------\n");
    printf("Jumlah mahasiswa : %d\n", jumlah_data);
    printf("Total nilai      : %.2f\n", total);
    printf("Rata-rata        : %.2f\n", rata_rata);
    printf("Nilai tertinggi  : %.2f (mahasiswa ke-%d)\n", nilai[idx_tertinggi], idx_tertinggi + 1);
    printf("Nilai terendah   : %.2f (mahasiswa ke-%d)\n", nilai[idx_terendah], idx_terendah + 1);
    printf("Jumlah lulus     : %d\n", jumlah_lulus);

    // Input nilai yang dicari
    printf("\nMasukkan nilai yang dicari: ");
    scanf("%lf", &target);

    // Menampilkan hasil pencarian
    printf("\nHASIL PENCARIAN\n");
    printf("------------------------------------------------\n");
    tampilkan_posisi_target(nilai, jumlah_data, target);
    int frekuensi = hitung_frekuensi(nilai, jumlah_data, target);
    printf("Jumlah kemunculan : %d\n", frekuensi);
    printf("================================================\n");

    return 0;
}

// Definisi Fungsi

void baca_nilai(double nilai[], int jumlah_data)
{
    int i;
    for (i = 0; i < jumlah_data; i++)
    {
        do
        {
            printf("Nilai mahasiswa ke-%d: ", i + 1);
            scanf("%lf", &nilai[i]);
            if (nilai[i] < 0.0 || nilai[i] > 100.0)
            {
                printf("Nilai tidak valid (harus 0-100).\n");
            }
        } while (nilai[i] < 0.0 || nilai[i] > 100.0);
    }
}

double hitung_total(const double nilai[], int jumlah_data)
{
    double total = 0.0;
    int i;
    for (i = 0; i < jumlah_data; i++)
    {
        total += nilai[i];
    }
    return total;
}

double hitung_rata_rata(const double nilai[], int jumlah_data)
{
    if (jumlah_data == 0)
        return 0.0;
    return hitung_total(nilai, jumlah_data) / jumlah_data;
}

int cari_indeks_tertinggi(const double nilai[], int jumlah_data)
{
    int idx_max = 0;
    int i;
    for (i = 1; i < jumlah_data; i++)
    {
        if (nilai[i] > nilai[idx_max])
        {
```

Tangkapan layar kompilasi dengan warning aktif

```
PS C:\Users\Administrator\Documents\Praktikum_Dasar_Pemrograman\Modul_10> gcc nilai_mahasiswa.c -o nilai_mahasiswa.exe
```

Tangkapan layar hasil eksekusi

```
Masukkan jumlah mahasiswa 1-50: 1
Nilai mahasiswa ke-1: 60

HASIL REKAP
------------------------------------------------
Jumlah mahasiswa : 1
Total nilai      : 60.00
Rata-rata        : 60.00
Nilai tertinggi  : 60.00 (mahasiswa ke-1)
Nilai terendah   : 60.00 (mahasiswa ke-1)
Jumlah lulus     : 1

Masukkan nilai yang dicari: 80

HASIL PENCARIAN
------------------------------------------------
Nilai 80.00 tidak ditemukan.
Jumlah kemunculan : 0
================================================
```

Tabel Pengujian

No.	Jumlah Mahasiswa	Data Nilai	Nilai Dicari	Expected Result	Actual Output	Status
1	0	–	–	Input jumlah ditolak	Input jumlah ditolak	Sesuai
2	51	–	–	Input jumlah ditolak	Input jumlah ditolak	Sesuai
3	1	60	60	Rata-rata 60; lulus 1; ditemukan mahasiswa ke-1	Rata-rata 60; lulus 1; ditemukan mahasiswa ke-1	Sesuai
4	5	80,60,90,60,75	60	Frekuensi 2; posisi ke-2 dan ke-4	Frekuensi 2; posisi ke-2 dan ke-4	Sesuai
5	3	0,0,0	0	Tertinggi & terendah 0; frekuensi 3	Tertinggi & terendah 0; frekuensi 3	Sesuai
6	3	100,0,50	50	Tertinggi 100; terendah 0; target ditemukan	Tertinggi 100; terendah 0; target ditemukan	Sesuai
7	3	70,80,90	99	Target tidak ditemukan	Target tidak ditemukan	Sesuai
8	50	Data valid 0–100	nilai valid	Program berjalan	Program berjalan	sesuai

				tanpa akses di luar batas	tanpa akses di luar batas	

Refleksi

Jelaskan manfaat array, fungsi, dan sequential search pada tugas ini.

1. Array memungkinkan penyimpanan sekumpulan data bernilai sejenis (seperti nilai mahasiswa) secara terstruktur dalam satu variabel sehingga mudah diakses menggunakan indeks.

2. Fungsi membantu memecah program menjadi modul-modul yang lebih kecil, terorganisir, dan reusable, serta memudahkan pembacaan dan pemeliharaan kode (seperti pemisahan fungsi baca, statistik, dan pencarian).

3. Sequential search mempermudah proses pencarian elemen data tanpa perlu mengurutkannya terlebih dahulu, serta memungkinkan pencarian seluruh lokasi kemunculan data secara berurutan.

X. Kesalahan Umum dan Standar Penulisan

Pelajari kembali bagian X pada Modul 8. Pastikan indeks berada pada rentang valid, jumlah_data tidak melebihi kapasitas, fungsi yang hanya membaca array menggunakan const, dan nilai -1 digunakan secara konsisten sebagai penanda tidak ditemukan.

Pemeriksaan Mandiri	Sudah / Belum
Indeks selalu 0 sampai jumlah_data - 1	Sudah
Tidak ada akses data[jumlah_data]	Sudah
jumlah_data tidak melebihi kapasitas array	Sudah
Accumulator diinisialisasi sebelum digunakan	Sudah
Array read-only menggunakan const	Sudah
Sequential search mengembalikan -1 jika tidak ditemukan	Sudah
Seluruh posisi/frekuensi ditampilkan sesuai kebutuhan	Sudah
Program dikompilasi tanpa warning penting	sudah

XII. Pernyataan Penggunaan AI

Pilihan	Pernyataan
Menggunakan AI	Saya menggunakan Gemini AI untuk membantu merancang skenario pengujian program, menyusun dokumentasi catatan debugging, serta memformulasikan refleksi pemahaman konsep modularisasi dan rekursi. Hasil tersebut saya periksa kembali melalui tracing, kompilasi, pengujian, dan perbandingan dengan ketentuan pada modul.